VERILOG CODE FOR UNIVERSAL SHIFT REGISTER

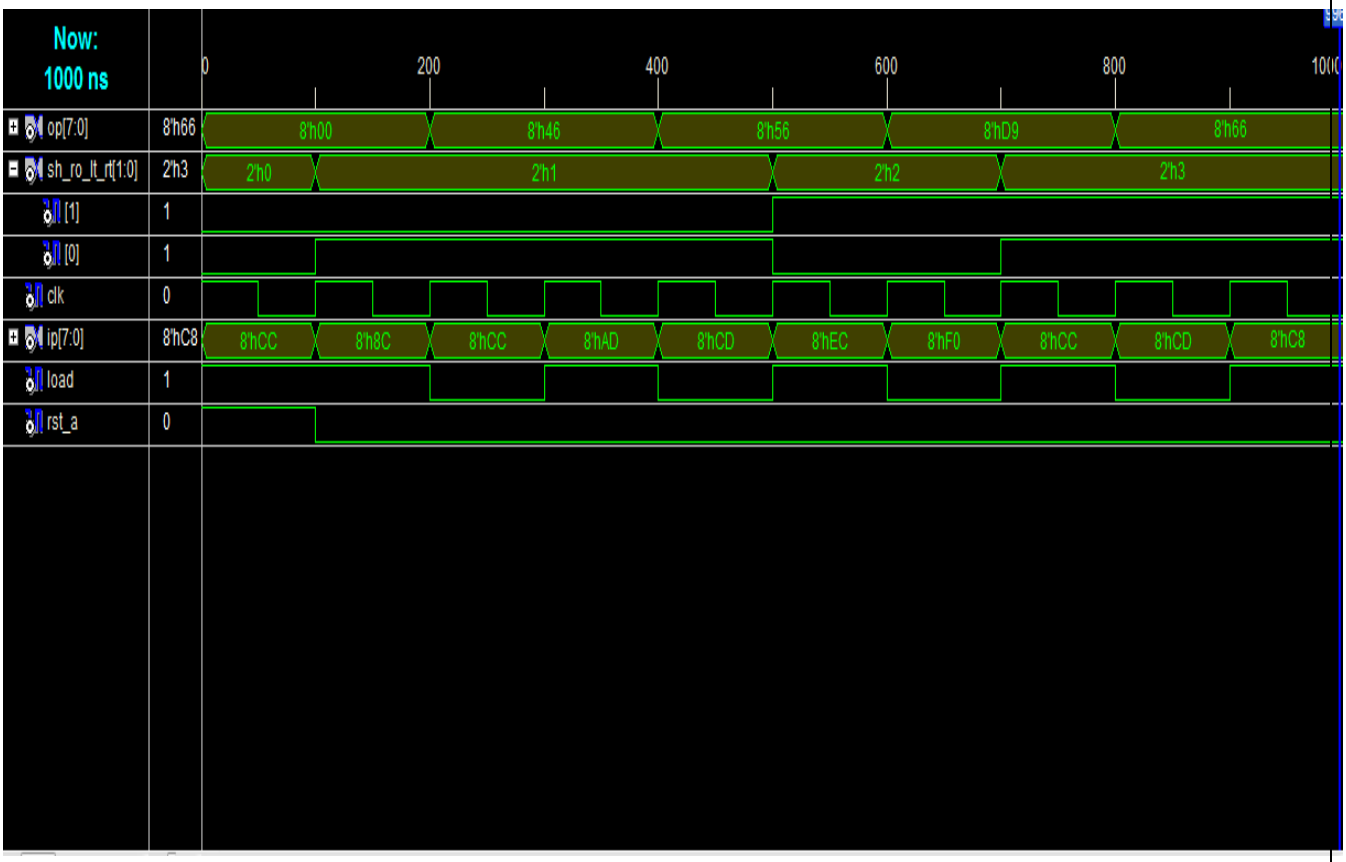
```
module uni_shift_8b(op,clk,rst_a, load,sh_ro_lt_rt,ip);
outputreg [7:0] op;
input load;
input [1:0] sh_ro_lt_rt;
input [7:0] ip;
inputclk, rst_a;
reg [7:0]temp;
always @(posedgeclk or posedgerst_a)
begin
if (rst_a)
op = 0;
else
case(load)
        1'b1:
begin                          //Load Input
temp = ip;
         //  op = temp;
end
        1'b0:                          //Operation
case (sh_ro_lt_rt)
           2'b00:  op = temp<<1;     //Left Shift
           2'b01:  op = temp>>1;     //Right Shift
           2'b10:  op = {temp[6:0],temp[7]}; //Rotate Left
           2'b11:  op = {temp[0], temp[7:1]};  //Rotate Right
default: $display("Invalid Shift Control Input");
endcase
default: $display("Invalid Load Control Input");
endcase
end
endmodule
```

Test Bench for Universal Shift Register

```
`timescale 1ns/1ps
module uni_shift_8b_tst;
reg [7:0] ip;
reg [1:0] sh_ro_lt_rt;
regload,rst_a,clk;
wire [7:0] op;
  uni_shift_8b u1 (.op(op), .ip(ip), .sh_ro_lt_rt(sh_ro_lt_rt), .load(load) , .rst_a(rst_a) , .clk(clk));
initial
begin
clk=1'b1;
forever #50 clk=~clk;
end
initial
begin
ip = 8'b11001100;
rst_a = 1'b1;
load = 1'b1;
sh_ro_lt_rt = 2'b00;
   #100;
ip = 8'b10001100;
rst_a = 1'b0;
load = 1'b1;
sh_ro_lt_rt = 2'b01;
   #100;
ip = 8'b11001100;
   load = 1'b0;
sh_ro_lt_rt = 2'b01;
   #100;
ip = 8'b10101101;
load = 1'b1;
sh_ro_lt_rt = 2'b01;
```

```
    #100;
ip = 8'b11001101;
load = 1'b0;
sh_ro_lt_rt = 2'b01;
    #100;
ip = 8'b11101100;
load = 1'b1;
sh_ro_lt_rt = 2'b10;
    #100;
ip = 8'b11110000;
load = 1'b0;
sh_ro_lt_rt = 2'b10;
    #100;
ip = 8'b11001100;
load = 1'b1;
sh_ro_lt_rt = 2'b11;
    #100;
ip = 8'b11001101;
load = 1'b0;
sh_ro_lt_rt = 2'b11;
    #100;
ip = 8'b11001000;
load = 1'b1;
sh_ro_lt_rt = 2'b11;
    #100;
    $stop;
end // initial begin
endmodule
```

SHIFT REGISTER OUTPUT WAVEFORM



VERILOG CODE FOR JK FLIP FLOP

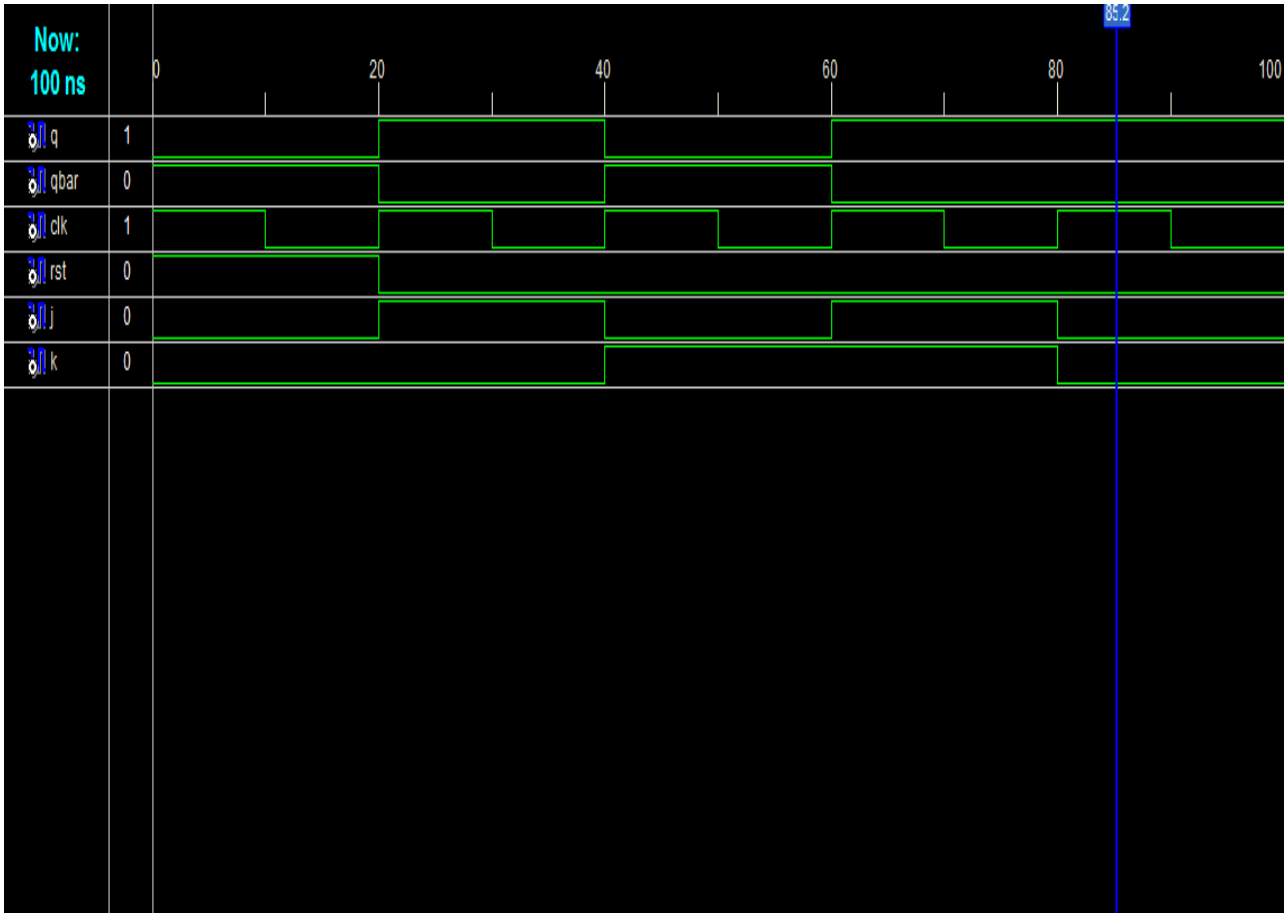
```
`timescale 1ns / 1ps
modulejk(j,k,clk,rst, q,qbar);
inputj,k,clk,rst;
outputq,qbar;
        reg q;
        always @(posedgeclk)
        begin
        if(rst) q<=1'b0;
        else begin
        case({j,k})
        2'b00: q<=q;
        2'b01: q<=1'b0;
        2'b10: q<=1'b1;
        2'b11: q<=~q;
        endcase
        end
        end
        assignqbar=~q;
endmodule
```

Test Bench for JK Flip Flop

```
`timescale 1ns / 1ps
modulefftb_v;
        // Inputs
        reg j;
        reg k;
        regclk;
        regrst;
        // Outputs
        wire q;
        wireqbar;
        // Instantiate the Unit Under Test (UUT)
```

```
	jkuut (
		.j(j),
		.k(k),
		.clk(clk),
		.rst(rst),
		.q(q),
		.qbar(qbar)
	);
	initial begin
	clk=1'b1;
	forever #10 clk=~clk;
	end
		initial begin
		// Initialize Inputs
		j = 0;
		k = 0;
	rst = 1;#20
		j = 1;
		k = 0;
	rst = 0;#20
		j = 0;
		k = 1;
	rst = 0;#20
		j = 1;
		k = 1;
	rst = 0;#20
		j = 0;
		k = 0;
	rst = 0;#20
		$stop;
	end
endmodule
```

JK FLIPFLOP WAVEFORM



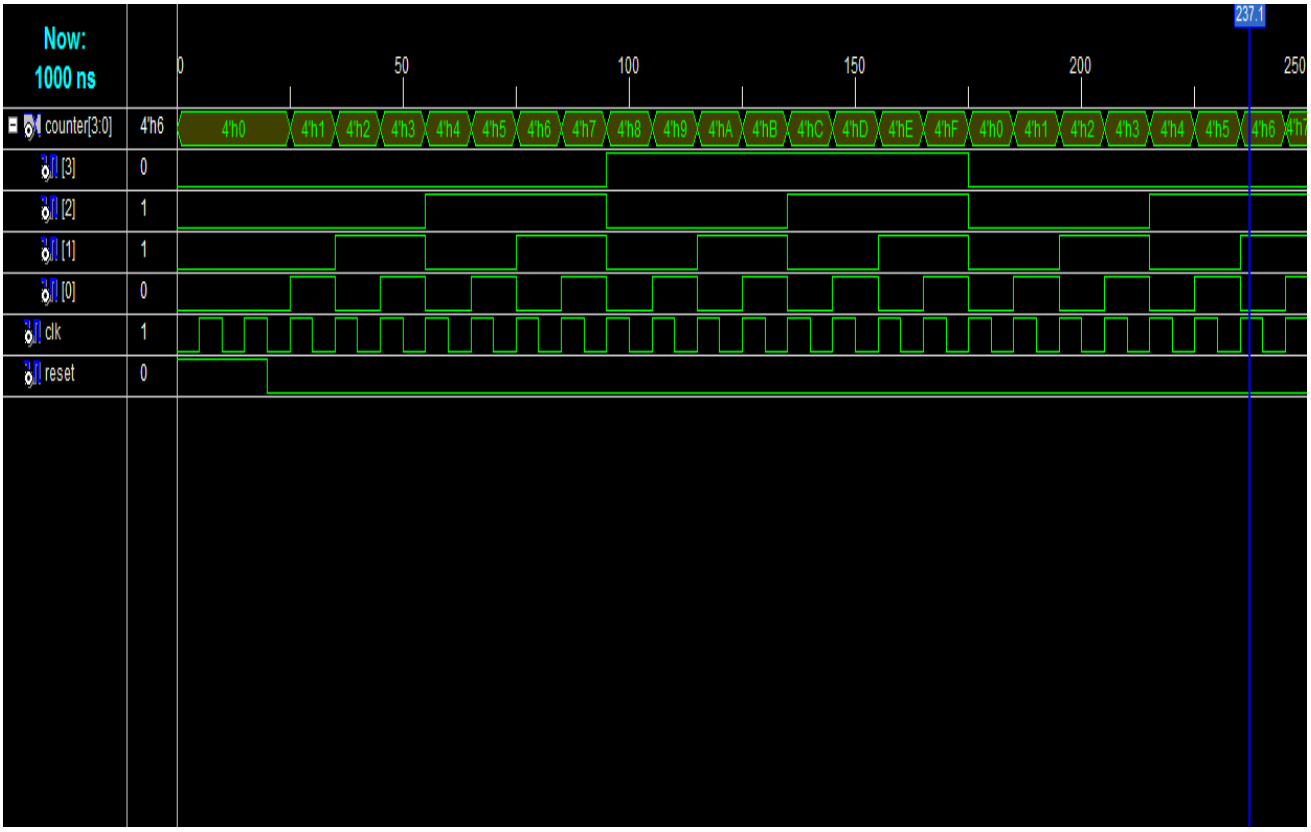
VERILOG CODE FOR UP COUNTER

```
moduleup_counter(input clk, reset, output[3:0] counter
   );
reg [3:0] counter_up;
// up counter
always @(posedgeclk or posedge reset)
begin
if(reset)
counter_up<= 4'd0;
else
counter_up<= counter_up + 4'd1;
end
assign counter = counter_up;
endmodule
```

Test Bench for Up Counter

```
moduleupcounter_testbench();
regclk, reset;
wire [3:0] counter;
up_counterdut(clk, reset, counter);
initial begin
clk=0;
forever #5 clk=~clk;
end
initial begin
reset=1;
#20;
reset=0;
end
endmodule
```

OUTPUT FOR SYN UP COUNTER



VERILOG CODE FOR RIPPLE COUNTER

```
moduleripple_carry_counter(q, clk, reset);
output [3:0] q;
inputclk, reset;
T_FF tff0(q[0], clk, reset);
T_FF tff1(q[1], q[0], reset);
T_FF tff2(q[2], q[1], reset);
T_FF tff3(q[3], q[2], reset);
endmodule

module T_FF(q, clk, reset);
output q;
inputclk, reset;
wire d;
D_FF dff0(q, d, clk, reset);
not n1(d, q); // not is Verilog-provided primitive. Case sensitive.
endmodule

module D_FF(q, d, clk, reset);
output q;
input d, clk, reset;
reg q;
always @(posedge reset or negedgeclk)
if (reset)
q = 1'b0;
else
q = d;
endmodule
```

Ripple Counter Test Bench

```
moduleripple_tb;
regclk; // Input
reg reset; // Input
wire [3:0] q; // Output

ripple_carry_counter r1 (.q(q), .clk(clk), .reset(reset));
initial
clk = 1'b0; // Set clk to 0
always
#5 clk = ~clk; // Toggle clk every 5 time units
initial
begin
reset = 1'b1;
#20 reset = 1'b0;
#180 reset = 1'b1;
#20 reset = 1'b0;
endendmodule
```

OUTPUT FOR RIPPLE COUNTER

